Video 05: Program execution

Complete script and teaching outline

Duration: 09:32. Video: BlueImpulse_Video05_Program_Execution.mp4.

Learning objectives

1. Trace fetch, decode, and execute
2. Give each register its job
3. Follow a four-instruction program

Guide pages: 12, 13, 14, 15, 39.

Teaching outline: Program execution

Open the guide at the pages below. Let learners answer before continuing through a thinking pause. Use the matching Python demonstration to change an input and explain the resulting change.

Time	Teaching step	Guide page	What to show or ask
00:00	How does a line of code become a result?	13	How does a line of code become a result?
00:35	Today’s route	13	Today’s route
01:13	Before the processor executes	14	A language may have both compiled and interpreted implementations.
02:01	Use the recipe analogy	13	PC: next instruction address; MAR: selected memory address; MDR: item moving to/from memory; CIR: current instruction; ACC: intermediate arithmetic
02:51	Fetch one instruction	13	A branch instruction may replace the normal next value of PC.
03:43	Read the four instructions	15	STORE copies 7 to address 200; it does not mean “store the number 200”.
04:27	Single-step the book’s program	15	Single-step the book’s program
05:34	Value or address?	15	After STORE 200: ACC = ? and memory[200] = ?
06:25	The next address can change	13	Normal next address; Branch selects another address; Instruction effects still matter
07:14	Change one instruction	15	LOAD #3; ADD #9; STORE 200
08:06	Instructions change machine state	13	Fetch → decode → execute; Each register has a job; Addresses differ from values; Trace: 3 + 4 → memory[200] = 7
09:01	Blue Impulse study guide	1	Blue Impulse study guide

The written tutorial PDF contains the full explanation and exact book excerpts. Numerical examples are teaching models, with assumptions stated in the guide, narration, and supporting file.

Complete narration

Read the narration verbatim. Bracketed pauses and on-screen prompts are teaching directions; keep the answer hidden until the indicated pause has finished. Timestamps match the source script.

00:00–00:35 · How does a line of code become a result?

Study guide page 13. Visual: hook.

Have you ever wondered what changes inside a computer after you press Run?

A program does not execute because the processor understands the whole page like a person reading an essay.

It follows encoded instructions, changing registers, memory, and the next instruction address as it works.

Open the Blue Impulse study guide at page thirteen, Fetch, decode, execute.

We will use that page and the tiny program on page fifteen to track every important state change by hand.

00:35–01:13 · Today's route

Study guide page 13. Visual: objectives.

You will explain fetch, decode, and execute, and give each teaching-model register a clear job.

You will trace four instructions and distinguish an immediate value from a memory address.

You will also connect source code, translators, and machine instructions without oversimplifying how modern language implementations work.

Welcome to the tutorial on program execution by Blue Impulse.

Have a small table ready with columns for the program counter, current instruction, accumulator, and memory location two hundred.

01:13–02:01 · Before the processor executes

Study guide page 14. Visual: book.

Start with page fourteen, From source code to execution. Machine code contains the instructions the processor executes.

An assembler translates assembly mnemonics and labels. A compiler produces another representation, often native code or bytecode.

An interpreter runs operations through a runtime, sometimes working with bytecode.

The same language can have compiled and interpreted implementations, and modern systems may combine several techniques.

Avoid the blanket claim that every interpreter translates exactly one source line at a time.

For our demonstration, the book supplies an invented assembly notation so that the state changes remain easy to see.

02:01–02:51 · Use the recipe analogy

Study guide page 13. Visual: compare.

Page thirteen uses a recipe to explain the register jobs. The program counter marks the next recipe line.

The memory address register selects the location. The memory data register carries the item read from memory.

The current instruction register holds the instruction being followed. The accumulator is a temporary mixing bowl for arithmetic.

We abbreviate those names as P C, M A R, M D R, C I R, and A C C.

The analogy helps you remember the jobs, but these are electronic storage locations inside a processor model.

Keep the jobs distinct: an address points to a location, while an instruction or value is the content being handled.

02:51–03:43 · Fetch one instruction

Study guide page 13. Visual: book.

Follow the numbered table on page thirteen. First copy the address in the program counter to the memory address register.

Read the instruction at that memory location into the memory data register.

Advance the program counter so it identifies the next instruction address, then copy the fetched instruction into the current instruction register.

The control unit can now decode the operation and its operand, and the processor executes the required work.

In this simplified model, every instruction occupies one word, so the program counter advances by one.

Real instruction sizes and exact timing depend on the architecture. Our table is a teaching model, not every processor's wiring diagram.

03:43–04:27 · Read the four instructions

Study guide page 15. Visual: book.

Turn to page fifteen. The program starts at address one hundred with LOAD, hash three.

The hash sign denotes an immediate value in this invented notation, so that instruction puts the number three into the accumulator.

Address one hundred and one contains ADD, hash four. Address one hundred and two contains STORE, two hundred.

Address one hundred and three contains HALT, which ends this teaching program.

The number two hundred in STORE identifies a memory location. It is not the value to put into the accumulator.

Before running anything, predict what that memory location will contain when the program finishes.

04:27–05:34 · Single-step the book's program

Study guide page 15. Visual: cpu-demo.

Let's single-step. Initially, the program counter is one hundred. Fetch the LOAD instruction and advance to one hundred and one.

Decode and execute LOAD, hash three. The accumulator now contains three.

Next fetch ADD, hash four from address one hundred and one. The program counter advances to one hundred and two.

Execution adds four to the accumulator's three, giving seven. Memory location two hundred has not been written yet.

Fetch STORE, two hundred. The program counter advances to one hundred and three, and execution copies seven into memory location two hundred.

The accumulator still contains seven. STORE copies the value; it does not turn the accumulator into an address.

Finally fetch HALT. In this simplified trace, the fetch advances the counter to one hundred and four before execution stops.

Our runnable Python model prints these same states and the register transfers, so you can check the trace against page fifteen.

05:34–06:25 · Value or address?

Study guide page 15. Visual: question.

Pause and check your prediction. After the STORE instruction, what are the accumulator and memory location two hundred?

Also ask yourself whether STORE, two hundred means storing the number two hundred.

[Pause for 5 seconds. Keep the answer hidden.]

The accumulator remains seven, and memory location two hundred receives seven. Two hundred is the destination address.

The hash sign in our LOAD and ADD operands marks literal values. The STORE operand has a different role.

This distinction prevents a common tracing mistake: confusing where data lives with the value stored there.

Use the book's instruction meanings before applying assumptions from a different programming language or processor.

On-screen question: After STORE 200: ACC = ? and memory[200] = ?

Answer after the pause: ACC = 7; memory[200] = 7

06:25–07:14 · The next address can change

Study guide page 13. Visual: diagram.

Return to page thirteen. It notes that a branch instruction may replace the normal next value of the program counter.

That is how the instruction sequence can follow a decision or repeat a loop instead of always moving forward.

Our four-instruction program contains no branch, so the normal next addresses stay easy to follow.

Page thirty-nine adds pipelining: real processors may overlap work on several instructions while preserving the required program behaviour.

Overlapping the implementation does not make the logical effects of a program arbitrary.

For a basic trace, first understand the defined instruction effects, then study how a specific processor implements them.

07:14–08:06 · Change one instruction

Study guide page 15. Visual: question.

Your turn. Change ADD, hash four to ADD, hash nine, while keeping LOAD, hash three and STORE, two hundred.

What value should reach memory location two hundred? Work through the accumulator before we reveal it.

[Pause for 6 seconds. Keep the answer hidden.]

LOAD gives three. Adding nine gives twelve. STORE copies twelve into memory location two hundred.

That is a small example of testing a prediction by changing one input to the program.

The included simulator accepts a different add value, prints the trace, and checks that the final memory result matches the arithmetic.

Try another value and explain each register's role aloud as the instruction moves through the cycle.

On-screen question: LOAD #3; ADD #9; STORE 200

Answer after the pause: memory[200] = 3 + 9 = 12

08:06–09:01 · Instructions change machine state

Study guide page 13. Visual: recap.

Recap the lesson. Source code reaches execution through a translator or runtime appropriate to its implementation.

The program counter identifies the next instruction. Fetch brings it from memory, decode interprets it, and execute performs its operation.

The memory address register selects a location, the memory data register carries data, and the current instruction register holds the operation.

Our teaching accumulator went from three to seven, and STORE copied seven to memory location two hundred.

Addresses and values have different roles. Branches can change the normal next instruction address.

Revisit pages thirteen through fifteen and complete the trace without watching the moving highlight. Understanding means predicting the state changes yourself.

09:01–09:32 · Blue Impulse study guide

Study guide page 1. Visual: credits.

This tutorial used the Blue Impulse study guide, especially pages twelve through fifteen, with the pipeline connection on page thirty-nine.

The original cover on screen credits the book we followed.

Use the companion tutorial PDF, complete narration, subtitles, and runnable instruction simulator for revision.

Blue Impulse study guide. Follow the instruction, predict the state, and check the result.